LIGHTYEAR

MS77 Declarative batch invocation

Status: **partial — built, awaiting an authorised environment**.

A declarative pack now names the jobs, input datasets, output comparisons and lane capabilities in scope. Its source lane observes completed jobs; it cannot submit or replay. This milestone is built and mock-tested, not proven against z/OS. The status must remain partial until a retained z/OS baseline exists.

```
PYTHONPATH=src python3 -m unittest tests.test_batch_pack tests.test_zosmf_reader
tests.test_zosmf_adapter -v
PYTHONPATH=src python3 -c "from lightyear_workflow.batch_pack import load, plan;
print(plan(load('spec/batch-packs/carddemo-intcalc.pack.json')))"
PYTHONPATH=src python3 -m pytest tests/ -q
```

The reference pack declares INTCALC and POSTTRAN, three comparisons, and the existing decimal, padding and filler normalization identifiers. It is a worked CardDemo fixture, not a customer pack. Referencing a normalization does not grant approval, perform comparison, or establish equivalence. Set comparisons require a record key. Empty job/comparison collections and source submission verbs are refused. `plan()` rechecks capabilities and derives submission lists; the source list is empty because submission is not an admitted source capability.

Binding to the qualified collector

`lightyear_workflow.zosmf_reader.ZosmfReader` sits above the existing `ZosmfClient` and `ZosmfJobsAdapter`. **`src/lightyear_runtime/zosmf.py` is unchanged.** Source calls use its read-only GET interfaces only.

The reader chooses the uniquely latest completed run by `exec-ended`, not by list position or job-ID ordering. Its default owner filter is `*`; supply an owner to narrow the search. An ambiguous latest run, missing completion time or possibly truncated listing requires an explicit job ID. The selected job must match the mapping, have status OUTPUT and retain its return code. A nonzero or ABEND return code is preserved, never converted into success. Failed selection clears the previous job context.

The existing client exposes JES spool retrieval, not a catalogued MVS dataset read operation. This binding therefore supports only **explicitly mapped spool outputs** of the selected job. Pass bindings separately from the qualified graph mapping, for example:

```
reader = ZosmfReader(client, config, mapping_path, datasets={
    "CARDDEMO.TRANFILE": {"ddname": "TRANSACT", "stepname": "STEP15"},
    "CARDDEMO.ACCTFILE": {"ddname": "ACCTFILE", "stepname": "STEP15"},
})
```

These reader bindings leave the graph mapping and its schema unchanged. Add `procstep` when needed to disambiguate a procedure step. A dataset must resolve to exactly one spool file with the selected job's identity and a known record count within the 5,000-record read limit. Missing, ambiguous, oversized or unbound outputs fail rather than becoming empty data. The example above is exercised with mock spool outputs; it does not assert that the existing CardDemo jobs actually retain these datasets in spool. The checked-in INTCALC collector mapping remains unchanged and contains no such bindings. POSTTRAN needs its own verified graph/output mapping before collection. Historical catalogued datasets, generation selection and binary dataset capture remain unimplemented; reading today's dataset would not prove what a past job consumed.

Evidence provenance

The reader returns the collector bundle and its `evidence_class`. The source receipt carries both through. HTTPS alone never upgrades simulated evidence; `attest_real_zos=True` remains explicit and

the existing non-loopback HTTPS gate is enforced. A boolean is required. These controls implement the collector's attestation contract; they do not independently prove that a host is z/OS.

Target receipts retain each lane's evidence class. Their combined class cannot exceed either lane: simulated source or target means **simulated**. Unlabelled fixture readers default to simulated; unknown classes are refused. Tests of the explicit `zos_observed` branch use a stub and are propagation tests only, never a z/OS execution receipt or a qualified baseline.

Supplied files and integration corrections

The supplied pack is retained byte-for-byte. The twelve supplied tests retain their assertions, with repository imports and fixture paths adjusted. The supplied module was placed rather than regenerated, with focused corrections because it originally dropped evidence provenance and hard-coded `source_submits` despite the spec's requirements. Additional tests cover these corrections, read-only mock HTTP collection, latest-run selection, missing/ambiguous output bindings, completion and identity checks, and simulation propagation through both lanes.

Boundaries

- No pack has run against a real z/OS system and no customer pack exists.
- `TargetLane.replay` requires a caller-supplied submit callable; no target submission implementation, credential provisioning or scheduler is built.
- Output declarations do not execute sequence, set or summary comparisons. No end-to-end batch equivalence verdict or qualification is produced.
- Inputs are declared dataset names, not captured historical input bytes. Their transfer and identity binding to target execution remain future work.
- Online transactions and CICS are entirely out of scope. Batch only.
- No workload, platform or milestone status is promoted to proven or done by these mock tests. Any technical-deck claim that batch invocation is done remains incorrect until the authorised z/OS baseline and remaining execution capabilities are demonstrated.